



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2023/2024

Modelação e Simulação de Sistemas Naturais
Relatório: Trabalho Prático 1

Turma: 31D

Grupo: 30

Nome: Miguel Alcobia

Número: 50746

Nome: Fábio Pestana

Número: 50756

Professor: Paulo Vieira

Data: 21 de Outubro de 2023

Índice

Lista de figuras	II
Abreviaturas e símbolos	III
Objetivos	1
Projeto 1 - Jogo da Vida e DLA	2
1. 1 - Jogo da Vida	2
1.1.1 - Cell	2
1.1.2 - GameOfLife	2
1.1.3 - TesteCA	3
1. 2 - DLA (Diffusion-Limited Aggregation)	4
1.2.1 - Walker	4
1.2.2 - DLA	5
Ensaio - Jogo da Vida de Conway	6
1. 1 - Origem dos autómatos celulares	6
1. 2 - John Conway	6
1. 3 - O Jogo da Vida e a Biologia	8
DLA - Diffusion Limited Aggregation	10
1. 1 - Definição	10
1. 2 - Fatores para a sua realização	10
1. 3 - A trajetória dos computadores com o DLA	10
1. 4 - Níveis de densidade urbana	12
Bibliografia	13

Lista de figuras

Figura 1 - Captura de ecrã do Jogo da Vida desenvolvido	4
Figura 2 - Em cima: DLA sem Stickiness. Em baixo, DLA com Stickiness (0.05)	5
Figura 3 - John von Neumann.....	6
Figura 4 - Stanisław Ulam.....	6
Figura 5 - Primeira página do artigo de Gardner	7
Figura 6 - Jonh Conway	8
Figura 7 - Tabela apresentada no artigo de 2016	9
Figura 8 - Árvore Browniana Circular	10
Figura 9 - Primeira página do artigo de A. K. Dewdney	11
Figura 10 - Estruturas resultantes de DLA, realizadas no estudo em 1989	12

Abreviaturas e símbolos

Lista de abreviaturas

DLA Diffusion-Limited Aggregation

Objetivos

Neste trabalho, os alunos tinham que desenvolver o Jogo da vida de Jonh Conway, criado em 1970, assim como desenvolver um algoritmo DLA (Diffusion-Limited Aggregation). Para ambos os desafios, os alunos teriam de ter por base os autómatos celulares lecionados nas aula. Após a exposição dos resultados obtidos no trabalho, apresentamos dois pequenos ensaios sobre os temas do trabalho.

Projeto 1 - Jogo da Vida e DLA

1.1 - Jogo da Vida

1.1.1 - Cell

Esta classe representa uma célula no Jogo da Vida, tem como atributos `linha`, `coluna` (onde ficam na *grid*), `state` (o seu estado), `previousState`, `vizinhos` (um array do tipo `Cell` que representa as células vizinhas).

O construtor da classe `Cell` recebe uma instância de `GameOfLife`, uma `linha` e uma `coluna` como parâmetros. Ele inicializa os atributos da célula com os valores iniciais e cria um array de `vizinhos` com base no valor do `raioVizinhanca`.

Além dos *gets* e *sets* esperados criamos os métodos:

- `display()`: Desenha a célula com cores diferentes, dependendo do estado atual e do estado anterior.
- `getAliveNeighbors()`: Conta o número de vizinhos vivos e retorna o resultado. Caso a célula estiver viva ele subtrai 1 a esse valor para esta não ser contada.

1.1.2 - GameOfLife

Esta classe é a responsável por simular o Jogo da Vida. Tem como atributos `linha`, `coluna`, uma matriz `cells` (representa as células do jogo); outra matriz, `Newcells` (representa o próximo estado das células após uma iteração), `cellAltura` e `cellLargura`, `raioVizinhanca` e `RuleSet`. O `RuleSet` é um `int`, responsável por atribuir o conjunto de regras a serem seguidas.

Além dos *gets* e *sets* esperados criamos os métodos:

- `createCells()`: Inicializa todas as células no *grid*.
- `setVizinhanca()`: calcula o número de vizinhos que uma célula tem e depois define quais são as células vizinhas para cada célula, através do método `setVizinhos()` que cada objeto `Cell` possui.
- `pixel2Cell()`: Converte coordenadas de pixel (`x` e `y`) em uma célula correspondente na *grid*.
- `rules()`: Dependendo do valor do inteiro `RuleSet` aplica o conjunto de regras para determinar o próximo estado de uma célula com base no número de vizinhos vivos e no estado atual da célula. Os conjuntos desenvolvidos foram: as regras clássicas de Conway (`RuleSet=1`, Variante 23/3), `HighLife` (`RuleSet=2`, Variante 23/36) e a Regra da maioria (`RuleSet=3`).

- `initRandom()`: Atualiza o estado das células com valores aleatórios (0 = morto ou 1 = vivo).
- `allCellStateZero()`: Atualiza o estado das células com valor 0 (células mortas).
- `update()`: Inicializa-se o array das novas células (próximo estado), obtém-se o valor dos vizinhos vivos para cada célula para depois aplicar-se as regras sobre esta, para depois guardar o seu estado e estado passado no novo array (para não se perder informação ao aplicar as regras em todas as células), no fim deste método atualiza-se o array `cells` através do array das novas `cells` e volta-se a definir a vizinhança para estas células.
- `display()`: Desenha todas as células do array `cells` a partir do método `display` de cada célula.

1.1.3 - TesteCA

Esta classe é uma implementação da interface `IProcessingApp()` e é onde é permitido controlar o Jogo da Vida. Tem como atributos `numLinhas` e `numColunas` (define os limites da grid do jogo), `check` (vê se a simulação está em execução (`true`) ou parada (`false`)). Também tem um atributo `ca`, uma instância da classe `GameOfLife`.

Além dos *gets* e *sets* esperados criámos os métodos:

- `setup()`: Cria uma instância da classe `GameOfLife` e define o *frameRate* e começa a simulação.
- `draw()`: É chamado a cada frame. A simulação é atualizada e exibida apenas se a variável `check` estiver a `true`.
- `keyPressed()`: Responde às teclas pressionadas pelo *user* e permite pausar/continuar a simulação (ao pressionar a *spacebar*), reiniciá-la com valores aleatórios (ao pressionar na tecla R) e definir regras diferentes (variante 23/3 - ao pressionar a tecla 3; variante 23/36- ao pressionar na tecla 6; regra da maioria - ao pressionar na tecla M).
- `mousePressed()`: Responde ao clique do rato, permitindo que o *user* altere o estado de uma célula selecionada na grid. Ele identifica a célula selecionada e altera seu estado entre vivo (1) e morto (0).

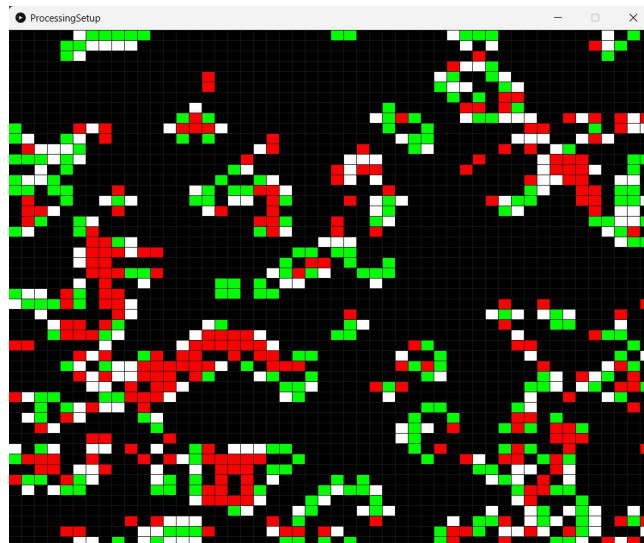


Figura 1 - Captura de ecrã do Jogo da Vida desenvolvido

As cores das células têm o seguinte significado:

- Verde = Célula que na geração passada estava viva e continua na atual.
- Vermelho = Célula viva na geração passada e morta na atual. (Vai morrer)
- Branca = Célula morta na geração passada e viva na atual. (Vai nascer)
- Preta = Célula que na geração passada estava morta e continua na atual.

Tal como mencionado anteriormente neste jogo da vida é aplicado três regras diferentes:

- Regra clássica - variante 23/3 : uma célula nasce se tiver 3 vizinhos vivos, morre se tiver menos de 2 ou mais de 3 e sobrevive se tiver 2 ou 3 vizinhos vivos.
- HighLife - variante 23/36: uma célula nasce se tiver 3 ou 6 vizinhos vivos, morre se tiver menos de 2 ou mais de 3 e sobrevive se tiver 2 ou 3 vizinhos vivos.
- Regra da maioria: se uma célula tiver metade ou mais vizinhos vivos ela nasce ou sobrevive, se não morre.

1. 2 - DLA (Diffusion-Limited Aggregation)

1.2.1 - Walker

Esta classe representa o que será uma partícula no DLA. Tem como atributos: `radius`, `pos` (posição), `state`, `type` (caracteriza o movimento), `Stickiness`. Este último define a probabilidade de um agente parar de "caminhar" e entrar no estado "STOPPED".

Utilizamos a "classe" especial `enum` para definir os estados necessários.

Além dos *gets* e *sets* esperados criámos os métodos:

- `updateState()`: Atualiza o estado da partícula com base na sua proximidade a outras partículas no estado "STOPPED" e na probabilidade definida por `Stickiness`.
- `wander()`: Move a partícula aleatoriamente no espaço (estado "WANDER"). A direção do movimento depende do tipo de partícula, que pode ser atraída para o centro (`type=1`), para a parte de baixo (`type=2`) ou para os cantos da janela (`type=3`).
- `display()`: desenha um círculo que representa o Walker.

1.2.2 - DLA

Esta classe é responsável pela simulação do DLA. Tem como atributos: `NUM_WALKERS`, `NUM_STEPS_PER_FRAME`, uma lista do tipo `Walker` chamada `walkers`, `level`, `NUM_MAX` e `nStoppedWalker`, que conta o nº de partículas no estado "STOPPED".

No método `draw()` é onde o loop do DLA é simulado e vai desenhando o movimento das partículas e sempre que um Walker pára outro será criado a menos que o `NUM_MAX` tenha sido atingido. No `setup()` é que é configurado o tipo de forma a ser criada no DLA, através do valor da variável `level`.

`level` tem 3 valores possíveis:

- 1 = *Point attractor*
- 2 = *Line attractor*
- 3 = *Rectangle attractor (atração para os cantos da janela)*

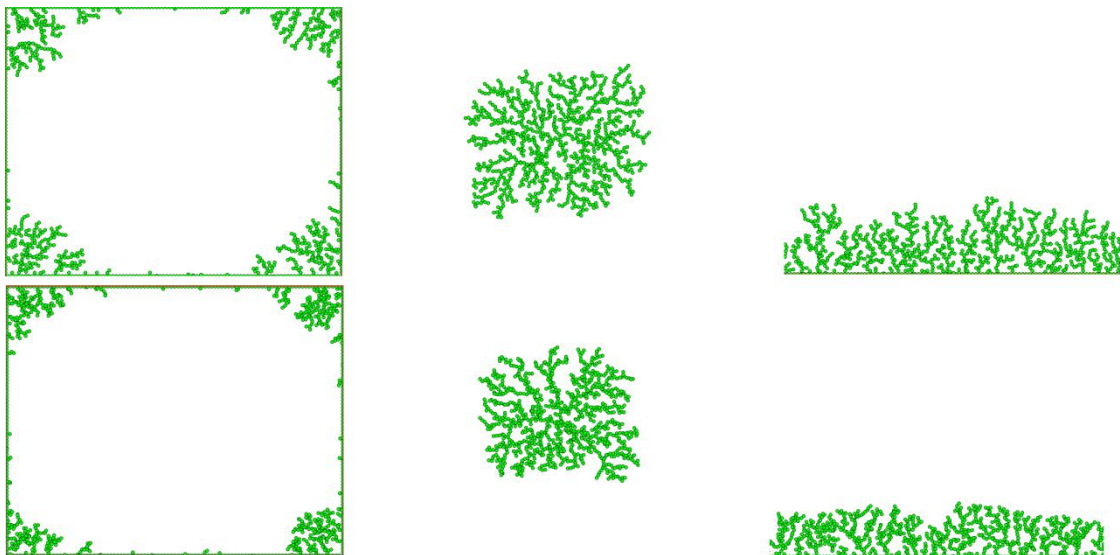


Figura 2 - Em cima: DLA sem Stickiness. Em baixo, DLA com Stickiness (0.05)

Na comparação dos DLAs apresentados na Figura 2, concluímos que quando o Stickiness é aplicado, criam-se figuras mais densas (com as partículas mais próximas umas das outras). Por outro lado, quando o Stickiness não é aplicado, as partículas ficam mais dispersas pela *grid*, pois a única condição que os une é o contacto com outras partículas, que já se encontram paradas num aglomerado.

Ensaio - Jogo da Vida de Conway

1. 1 - Origem dos autómatos celulares

Stanislaw Ulam , enquanto trabalhava no Laboratório Nacional de Los Alamos¹ na década de 1940, estudou o crescimento de cristais, usando uma rede simples como modelo. Ao mesmo tempo, John von Neumann, colega de Ulam em Los Alamos, estava a trabalhar no problema dos sistemas auto-replicantes. O projeto von Neumann tinha a intenção de fazer com que um robô construísse outro robô. Ao desenvolver este projeto, von Neumann percebeu a grande dificuldade de construir um robô com estas características e o grande custo que todo o projeto envolveria . Ulam foi quem sugeriu o uso de um sistema discreto para criar um modelo reducionista de auto-replicação. Os dois juntaram-se e criaram um método para calcular o movimento de líquidos no final dos anos 1950. O conceito base do método consistia em considerar um líquido como um grupo de unidades discretas e calcular o movimento de cada uma delas com base no comportamento das suas vizinhas. Assim nasceu o primeiro sistema de autómatos celulares.



Figura 3 - John von Neumann



Figura 4 - Stanislaw Ulam

1. 2 - John Conway

Motivado pela lógica matemática e pelo trabalho de Ulam sobre jogos de simulação, John Conway começou a fazer testes em 1968 com uma variedade de regras diferentes de autómatos celulares bidimensionais. O objetivo inicial de Conway era definir um autômato celular interessante e imprevisível. De acordo com Martin Gardner, Conway fez experiências com diferentes regras, procurando regras que permitissem que os padrões crescessem "aparentemente" sem limites, mantendo a dificuldade de provar que um determinado padrão o faria. .

¹ No âmbito de Projeto Manhattan, durante a Segunda Guerra Mundial

O jogo apareceu pela primeira vez ao público na edição de outubro de 1970 da revista *Scientific American*, na secção "Mathematical Games" de Martin Gardner, que se baseava em conversas pessoais com Conway. Teoricamente, o Jogo da Vida tem o poder de uma máquina de Turing universal: qualquer coisa que possa ser calculada algoritmicamente pode ser calculada no Jogo da Vida. Gardner escreveu: "Devido às analogias do Jogo da Vida com a ascensão, queda e alterações de uma sociedade de organismos vivos, pertence a um grupo cada vez maior dos chamados 'jogos de simulação' (jogos que se assemelham a processos da vida real)."

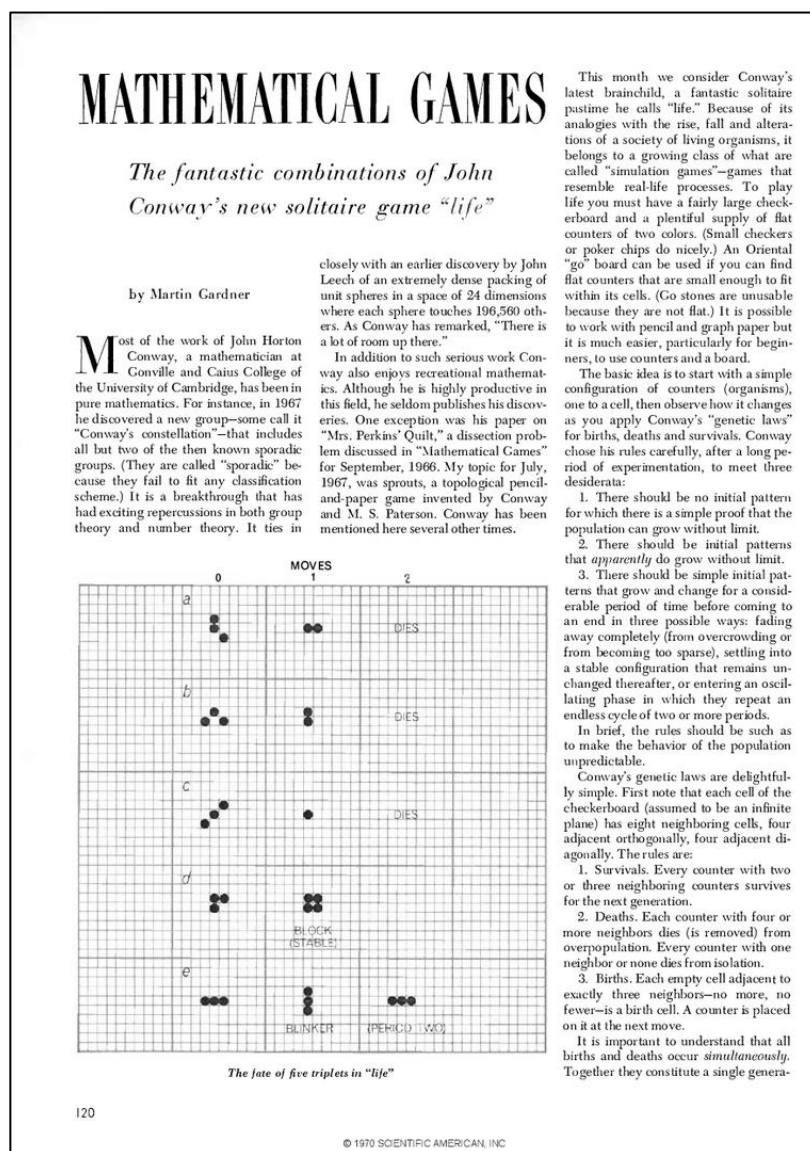


Figura 5 - Primeira página do artigo de Gardner

A popularidade do Jogo da Vida foi favorecida por surgir ao mesmo tempo que os computadores tornavam-se cada vez mais acessíveis. O jogo podia ser executado durante horas nestas máquinas, que de outra forma ficariam inutilizadas durante a noite. Neste aspeto, foi um prenúncio da popularidade posterior dos fractais gerados por computador. Para muitos, o Jogo da Vida era simplesmente um desafio de programação: uma forma divertida de utilizar ciclos de CPU que de

outra forma seriam desperdiçados.



Figura 6 - Jonh Conway

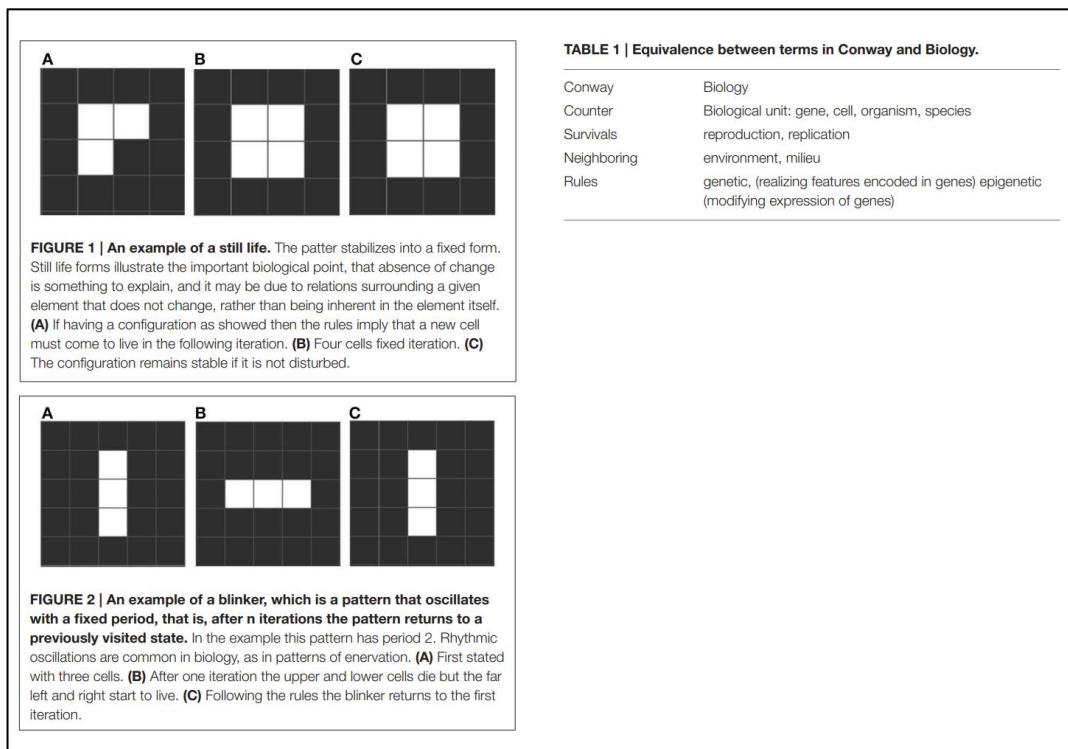
1.3 - O Jogo da Vida e a Biologia

Num artigo publicado em 2016 na *Frontiers in Cellular and Infection Microbiology*, intitulado como “Conway's “Game of Life” and the Epigenetic Principle”, é feita um paralelismo interessante entre o jogo de Conway e a própria Biologia.

Para as células existentes, existem apenas três estados possíveis: Sobrevivem, Morrem ou Nascem, como mostram as regras do jogo:

- Qualquer célula viva com dois ou três vizinhos vivos **sobrevive**.
- Qualquer célula morta com três vizinhos vivos torna-se uma célula viva (**Nasce**).
- Todas as outras células vivas **morrem** na próxima geração e todas as outras células já mortas permanecem mortas.

A situação simulada pelo *Life* pode ser encontrada em muitos sistemas biológicos. Tendo isto em vista, os autores do artigo elaboraram uma tabela que põe lado a lado os termos usados por Conway ao falar do seu jogo e os termos equivalentes na Biologia:



DLA - Diffusion Limited Aggregation

1. 1 - Definição

Os agregados (clusters) criados no DLA, são muitas vezes chamados de árvores brownianas.

Este nome surge do movimento efetuado pelas partículas no algoritmo, o movimento browniano, que por sua vez tem o nome derivado de Robert Brown (1773 - 1858) um botânico e paleobotânico escocês.

Estas árvores são estruturas dendríticas, ou seja, resultam de várias ramificações, neste caso estão associados ao processo da difusão.

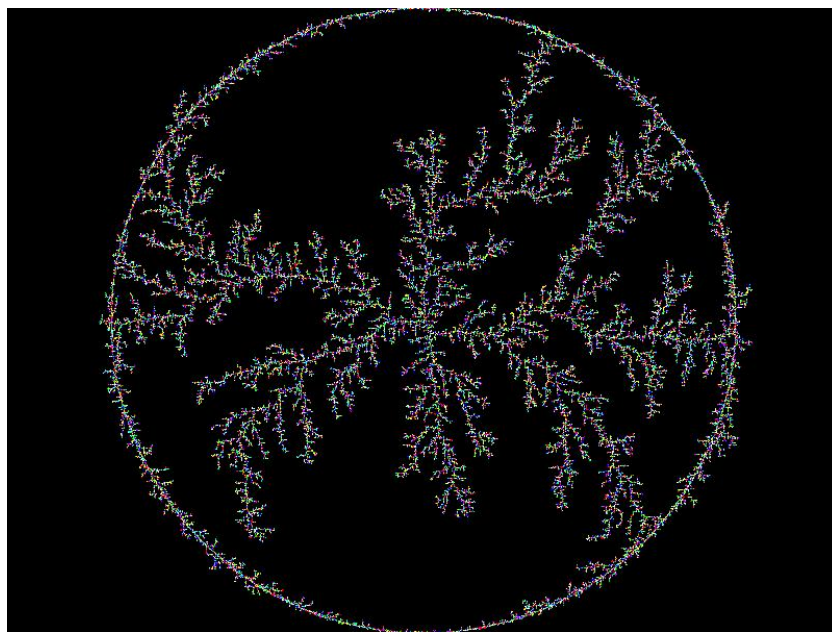


Figura 8 - Árvore Browniana Circular

1. 2 - Fatores para a sua realização

Qualquer algoritmo de DLA depende dos seguintes fatores para trabalhar corretamente:

- Uma posição para a sua partícula estacionária (*seed*),
- Uma posição para as partículas futuras (pode ser uma posição aleatória, no topo do ecrã etc.)
- Um algoritmo para a movimentação das partículas (muitas vezes é aleatório, mas o factor que Witten e Sander escolheram de excluir uma partícula quando esta tocava no limite do ecrã é um exemplo das alternativas existentes)

1. 3 - A trajetória dos computadores com o DLA

Ao lançarem o artigo em 1981, Witten e Sander deram origem a um enorme movimento que rapidamente popularizou o algoritmo, com outros artigos a serem lançados nessa mesma década, sendo um dos mais populares o artigo de Alexander Keewatin Dewdney, em 1988 para a *Scientific*

COMPUTER RECREATIONS

*Random walks that lead
to fractal crowds*



by A. K. Dewdney

"The most useful fractals involve chance... both their regularities and their irregularities are statistical."

—BENOIT B. MANDELBROT,
The Fractal Geometry of Nature

Is it a zinc ion, a bit of soot or a staggering drunk wandering about on the display screen? Does it join others of its kind to form a metallic flower, a dark patch or an insensible crowd? The answer depends only on one's imagination, but—flower, patch or crowd—the shape of the aggregate structure is unquestionably fractal; its fractal dimension (which is between 1 and 2) can even be measured.

A fractal, such as the one in the illustration on the opposite page, is created by a program called SLO GRO. It directs a succession of particles to wander aimlessly about the screen until they eventually encounter a collection of stationary kindred. As soon as the encounter takes place, the particle's motion is frozen. As several hundred particles accumulate one by one in this way, the collection slowly develops the irregular branches and tendrils characteristic of fractal shapes.

The SLO GRO program was inspired indirectly by an article that appeared in this magazine two years ago [see "Fractal Growth," by Leonard M. Sander; *SCIENTIFIC AMERICAN*, January, 1987]. The article described the particular growth process I have outlined above, which goes by the name of diffusion-limited aggregation, or DLA.

Several intrepid souls with the kind of adventurous spirit that characterizes this department's readership did not wait for me to pick up on DLA-simulation programs but wrote versions of SLO GRO for themselves. In particular, my hand has been forced by Eric M. Smiertka of Santa Clara, Calif. With little more than an elementary programming course under his

belt, he wrote a successful version of SLO GRO. Not content with merely watching his fractal grow, he measured its dimension as well.

The DLA process might well represent the diffusion of zinc ions through a two-dimensional electrolytic solution. When ions come in contact with an aggregation of metallic zinc at an electrode, they immediately bond with their atomic mates in the aggregate. Particles of soot that are wafted hither and yon before meeting other soot particles clinging to some substrate also seem to follow the same process, although in their case the cohesion is electrostatic in nature.

The most colorful (albeit the least realistic) model for a DLA process involves a succession of drunks wandering about in the dark until they stumble on a crowd of insensate comrades; lulled by the sounds of peaceful snoring, they instantly lie down to sleep. An aerial view of the slumbering crowd by morning light might well reveal the same fractal shape found in a zinc cluster or a soot patch.

The key to modeling the diffusion part of the DLA process is a "random walk." Each step of a random walk in two dimensions is taken in one of four randomly selected directions: north, south, east and west. (Sometimes a random walk also allows intermediate directions such as southeast, but I shall stick with the more restrictive version.) The top illustration on page 118 outlines a typical path made by a particle on a random walk. The particle seems to meander aimlessly. Watching such a particle on the screen, one can be excused for having serious doubts that it will ever reach a given destination. But the particle does have a goal (to be somewhat teleologic): to reach the crowd of fellow particles accumulating at the center of the screen.

It all starts when SLO GRO places a single particle at the center of the computer screen. A particle is then set on a random walk from a random point on a large circle centered on the initial particle. After a long and arduous journey the particle happens on the stationary particle and stops. As soon as that happens SLO GRO will dispatch another particle on a similar journey from another random point on the circle. Particle after particle collects at the site of the original one and a strange shape emerges within the circle: a tree-like growth with oddly twisted branches and twigs. The shape results from the tendency of a randomly walking particle to run into an outer part of the crowd long before it encounters a cohort much deeper in the crowd; twigs are more likely to grow from branches than from the core, so to speak.

All of this takes time. As Smiertka says, "It takes four or five hours to run the program on my IBM XT, so I just let it run overnight." The algorithms described below give rise to a somewhat faster version of the program, but of course the duration of a run depends on how many particles one wants to accumulate.

The diffusion algorithm (SLO) is a bit easier to write than the aggregation one (GRO). For the sake of simplicity I shall confine all the action to the inside of a circle 200 pixels in diameter. To choose the point on the circle from which a given particle begins its random walk, SLO computes a random angle in one instruction and then in the next two instructions works out the coordinates of the point on the circle that defines that angle:

```
angle ← random × 360
x ← 100 × cosine(angle) + 100
y ← 100 × sine(angle) + 100
```

The computer itself must choose a random number labeled *random*. Because such a number lies between 0 and 1, it must be multiplied by 360 to yield a random angle between 0 and 360 degrees. (For systems that employ radians instead of degrees, *random* must be multiplied by 2π , or approximately 6.283.) The next step calculates the x coordinate of the point on the circle that lies at that angle by multiplying 100 (the circle's radius) by the cosine of the angle. The third step works out the y coordinate of the point with the sine function. Because most popular programming systems place the origin—the point with coordinates (0,0)—in one corner of the screen, I have added offsets of 100 to

Figura 9 - Primeira página do artigo de A. K. Dewdney

Contudo, os computadores na altura levavam horas, às vezes dias, para chegar a uma pequena árvore. A situação só melhorou não com o avanço da tecnologia, mas também com a apresentação do algoritmo Brady-Ball (1985), como conta Sander num artigo, em julho de 2000, para a *Contemporary Physics*. Sander relata que a possibilidade de DLA com mais partículas começou-se a notar uma distorção na estrutura (*lattice*) e que durante um verão teve a oportunidade de trabalhar

com Robin Ball e conseguiram provar que realmente existia uma distorção para *clusters* muito grandes (com aproximadamente 10^6 partículas).

1. 4 - Níveis de densidade urbana

O DLA também pode ser utilizado para fazer estudos bastante interessantes que saem do núcleo informático. Em 1989, no artigo *Diffusion-limited aggregation and the fractal nature of urban growth*² foi realizado um estudo sobre os níveis de densidade urbana.

Acreditava-se que as densidades residenciais diminuíssem à medida que a distância aumentava em relação ao centro de uma cidade. As explicações na altura para este evento eram o declínio do valor dos terrenos e da densidade das redes de transportes e o aumento da dimensão das parcelas de terreno à medida que a distância à cidade aumenta. Contudo, naquela época já existiam estudos que se opunham a esta ideia e foi onde o DLA se revelou útil.

As várias análises feitas aos processos de DLA, mostravam que estes níveis podiam surgir naturalmente, pois mesmo com variações no valor dos terrenos, nas redes de transportes e no tamanho das parcelas, as densidades urbanas ainda podem diminuir à medida que a distância do centro urbano aumenta devido ao processo de crescimento descrito pelo DLA.

Em suma, com o aumento da distância ao centro a densidade populacional descenderá sempre, sendo resultado de uma ação natural.

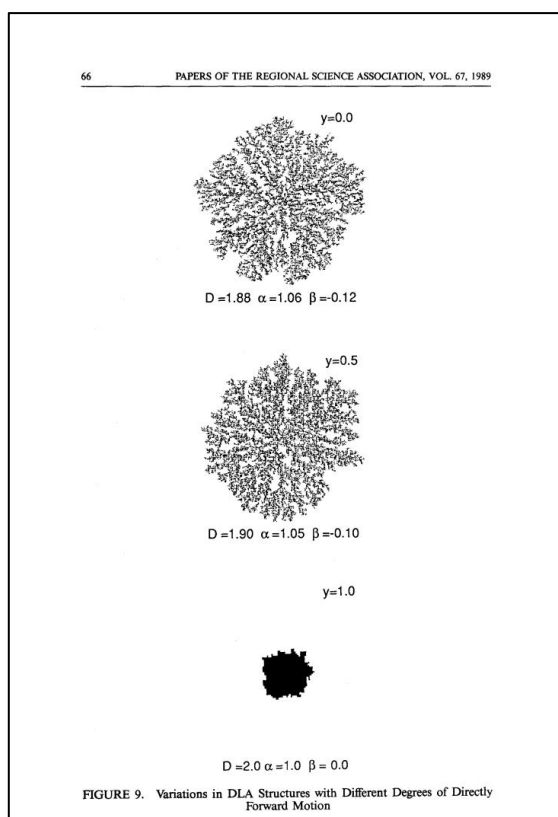


Figura 10 - Estruturas resultantes de DLA, realizadas no estudo em 1989

² A. Stewart Fotheringham, Batty, M., & Longley, P. A. (1989).

Bibliografia

Consulta:

- Vieira, Paulo. (2023). *SISTEMAS DINÂMICOS DISCRETOS DE ORDEM SUPERIOR* (via Moodle)
- Haber, D. (2022, December 27). *nlife-color*. GitHub. <https://github.com/DougHaber/nlife-color>
- *2D Cellular Automata: Majority Rule*. (n.d.). Retrieved October 22, 2023, from <https://understandingthecomplex.blogspot.com/2016/06/2d-cellular-automata.html>
- Webb, J. (2020, October 17). *Simulating 2D diffusion-limited aggregation (DLA) with JavaScript*. Medium. <https://medium.com/@jason.webb/simulating-dla-in-js-f1914eb04b1d>
- *DLA - Diffusion Limited Aggregation*. (n.d.). Paulbourke.net. Retrieved October 22, 2023, from <https://paulbourke.net/fractals/dla/>

Ensaaios:

Game of Life:

- Wikipedia Contributors. (2023, October 19). *Conway's Game of Life*. Wikipedia; Wikimedia Foundation. https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life#Origins
- Gardner, M. (1970). MATHEMATICAL GAMES. *Scientific American*, 223(4), 120–123. <http://www.jstor.org/stable/24927642>
- Caballero, L., Hodge, B., & Hernandez, S. (2016). Conway's "Game of Life" and the Epigenetic Principle. *Frontiers in Cellular and Infection Microbiology*, 6. <https://doi.org/10.3389/fcimb.2016.00057>

DLA:

- Witten, T. A., & Sander, L. M. (1981). Diffusion-Limited Aggregation, a Kinetic Critical Phenomenon. *Physical Review Letters*, 47(19), 1400–1403. <https://doi.org/10.1103/physrevlett.47.1400>
- Sander, L. M. (2000). Diffusion-limited aggregation: A kinetic critical phenomenon? *Contemporary Physics*, 41(4), 203–218. <https://doi.org/10.1080/001075100409698>
- *Diffusion-limited aggregation*. (2021, July 3). Wikipedia. https://en.wikipedia.org/wiki/Diffusion-limited_aggregation#References
- Dewdney, A. K. (1988). Computer Recreations. *Scientific American*, 259(6), 116–119. <http://www.jstor.org/stable/24989313>

- A. Stewart Fotheringham, Batty, M., & Longley, P. A. (1989). *Diffusion-limited aggregation and the fractal nature of urban growth*. 67(1), 55–69. <https://doi.org/10.1007/bf01934667>